



Understanding GPUs

From the **device** to **RAPIDS** : how the pieces fit together

Jaya Venkatesh and Naty Clementi · NVIDIA

SciPy 2026 · GPU Deployment & Debugging Tutorial ·

Why bring data science to the GPU?

Bigger data

Datasets keep outgrowing what's comfortable on a CPU

Parallel by nature

Filtering, aggregating, and math over millions of rows are independent

Familiar APIs

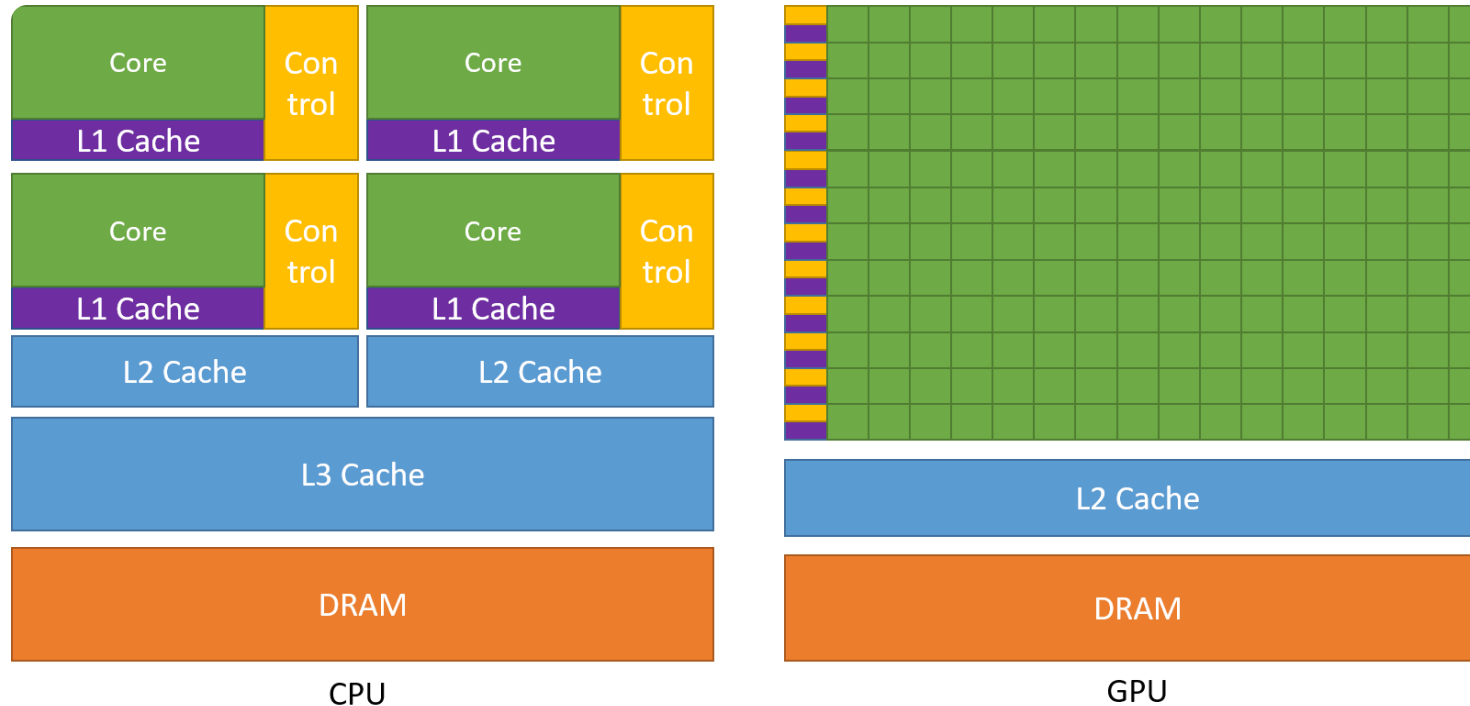
You can get there without leaving Python

But how does a GPU actually help you accelerate your code?

1 · The GPU vs the CPU

Both process data, just with a different philosophy

A CPU and a GPU are built for different jobs

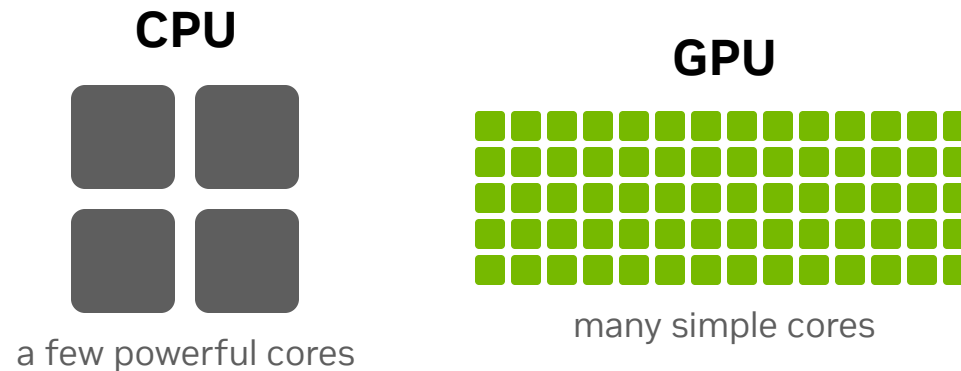


Source: NVIDIA CUDA C++ Programming Guide

CPU: handles **many different tasks**, finishing **each one fast** → *low latency*.

GPU: runs the **same task across lots of data** all at once → *high throughput*.

A few specialists vs many simple workers



- CPU cores are **few but powerful**, great at complex, branchy logic
- GPU cores are **many but simple**, built to all do the **same operation** on different data, together
- The more your work splits into **independent pieces**, the more **using the GPU pays off**

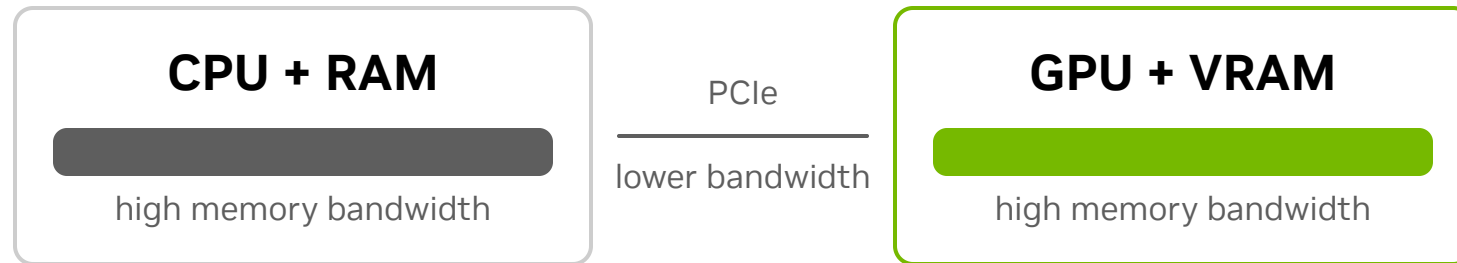
Two machines, two separate memories



- The GPU is a **separate device** with its **own memory**: it **cannot** read data sitting in system RAM
- Before the GPU can work on your data, that data must be **copied into the GPU's memory**

So step one of "run it on the GPU" is always: **get the data there.**

The slow part is getting data *across*



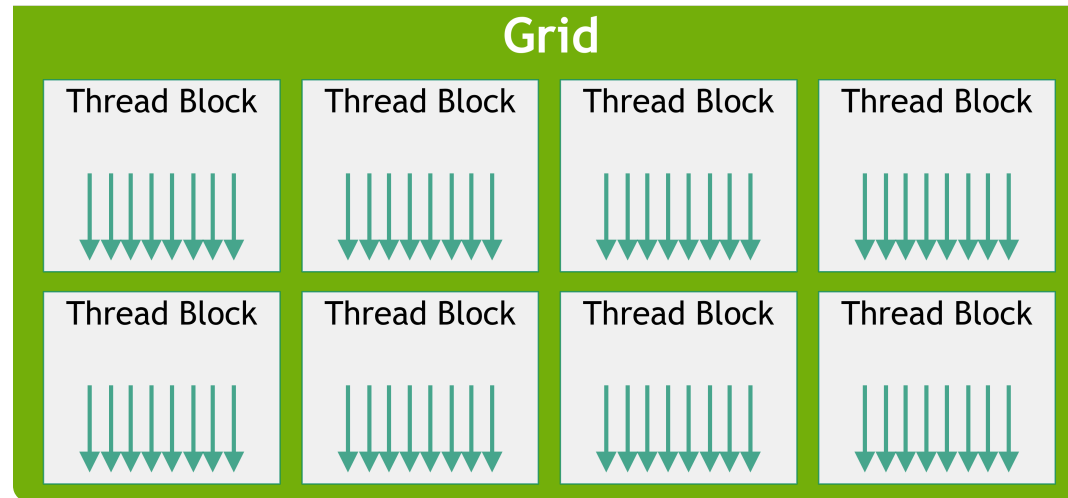
- **Inside** each device, **memory bandwidth is high**: RAM and VRAM are built to feed their cores fast
- The **interconnect between them (PCIe)** carries data at **far lower bandwidth**
- The expensive part often isn't the *computing*; it's the **host-to-device transfer** across PCIe

2 · How a GPU processes data

Many small tasks at once, not one big task in order

Parallel work: threads, blocks, grids

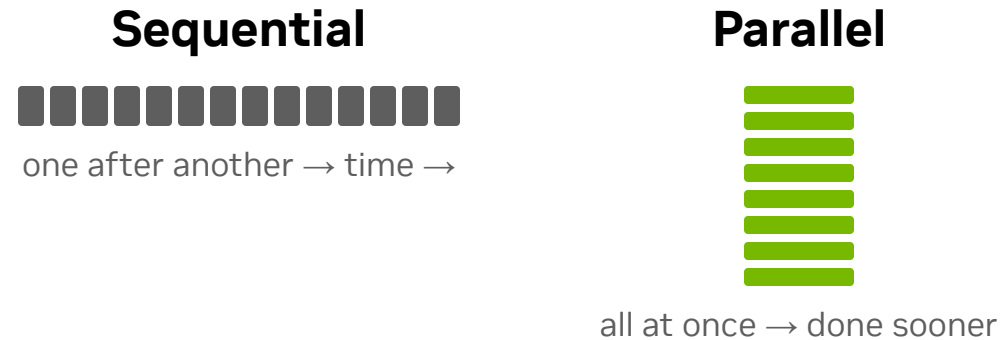
The GPU runs **one operation across thousands of elements at once**, one **thread** per item.



Source: NVIDIA CUDA C++ Programming Guide

- Threads are grouped into **blocks**; blocks together form a **grid**
- You describe the work *once*; the GPU runs it across the **whole grid in parallel**
- Perfect for **independent** work, not optimal for **sequential**, step-by-step logic

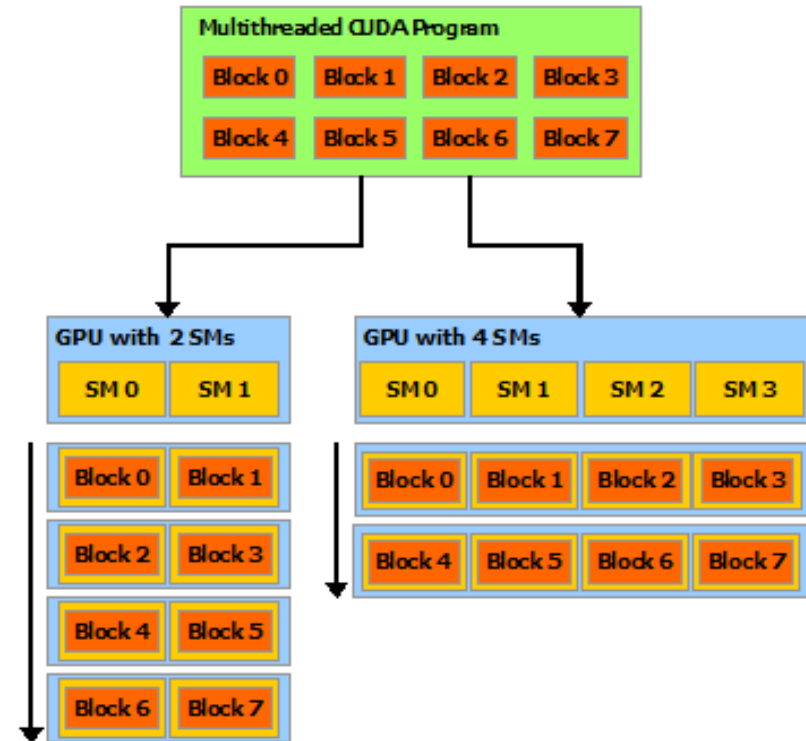
Why GPU is better for parallel than sequential



- When work items are **independent**, their order doesn't matter: item B doesn't need item A's result
- A GPU runs a **huge number of independent items at the same time**
- Step-by-step work that must run **in order** can't use that; this isn't where a GPU shines

SMs and warps: where the work runs

- A GPU is made of **Streaming Multiprocessors (SMs)**, its parallel engines
- Your **blocks are distributed across the SMs**; more SMs → more work at once, **automatically**
- Inside an SM, threads run in lock-step groups of **32, called a warp** (*SIMT: Single Instruction, Multiple Threads*)



Source: NVIDIA CUDA C++ Programming Guide

Compute-bound vs IO-bound

✓ Compute-bound: GPU shines

- A matrix multiply, training a model, transforming millions of rows
- **Lots of math** → the cores stay saturated

✗ IO-bound: GPU waits

- Reading a file off disk, waiting on a network call
- The bottleneck is **waiting**, not math

Adding compute units only helps when **computation** is the bottleneck. If you're waiting on data, more cores don't make the wait shorter.

So what actually fits a GPU?

✓ Parallel

The same operation over millions of elements

✗ Sequential

Each step depends on the previous one

✓ Compute-bound

Lots of math per byte of data

✗ IO-bound

Time spent waiting on disk / network

Takeaway: a GPU shines on big, math-heavy, parallel work, so the goal is to match your workload to its strengths.

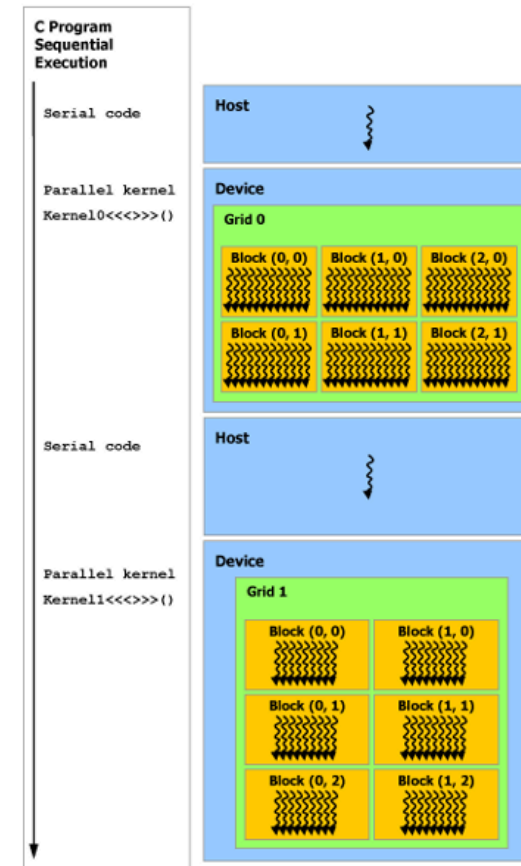
3 · CUDA

The bridge that lets software use the GPU

What is CUDA, and why C?

CUDA is NVIDIA's parallel-computing **platform + programming model**, built on **C/C++** for its low-level speed and control over the hardware.

- You write GPU code as **kernels** in **CUDA C/C++**: standard C/C++ plus a few extensions, compiled by `nvcc`
- The **host** (CPU) runs serial code and **launches kernels** on the **device** (GPU)
- Serial host code and parallel kernels **alternate** (see diagram →)



Source: NVIDIA CUDA C++ Programming Guide

A CUDA kernel is just C, with a twist

```
// __global__ marks a function that runs ON the GPU, once per thread
__global__ void add(float *a, float *b, float *c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // which element am I?
    c[i] = a[i] + b[i];                             // one thread, one element
}

// The host launches it across a grid: <<< blocks, threads-per-block >>>
add<<<blocks, threads>>>(a, b, c); // every element added in parallel
```

The `__global__` keyword and the `<<< >>>` launch syntax are the C extensions CUDA adds. A sum, a sort, a join all become kernels like this, but you'll rarely write one: the layers above ship them ready-made.

4 · CUDA Python & the CUDA Toolkit

The building blocks above raw CUDA

The CUDA Toolkit: the foundation libraries

Everything above the driver is **built on the Toolkit**. It provides:

Compiler & runtime

nvcc, libcudart, headers

Math libraries

cuBLAS, cuFFT, cuSPARSE, cuSOLVER, cuRAND

Communication

NCCL, NVSHMEM for multi-GPU

Why it's central

cuDF, cuML & CuPy don't reinvent math, they call these

These hand-tuned libraries are the **ready-made kernels**: the sorts, joins, and math behind your data work come straight from here.

CUDA Python: pick the right door

You don't have to write C++ to use CUDA. Match the door to the need:

CuPy

"I have NumPy code" → swap in GPU arrays

Numba (CUDA)

"I need a custom kernel" → write it in Python

cuda.core / cuda.bindings

"I'm building a library / need low-level control"

All three reach the **same CUDA underneath**. RAPIDS sits on top of them, so most of the time you won't pick a door at all.

5 · RAPIDS & CUDA-X

Predefined kernels for data science

CUDA-X & RAPIDS: kernels you don't have to write

CUDA-X

NVIDIA's library collection on top of CUDA: math, deep learning (cuDNN, TensorRT), comms, data science

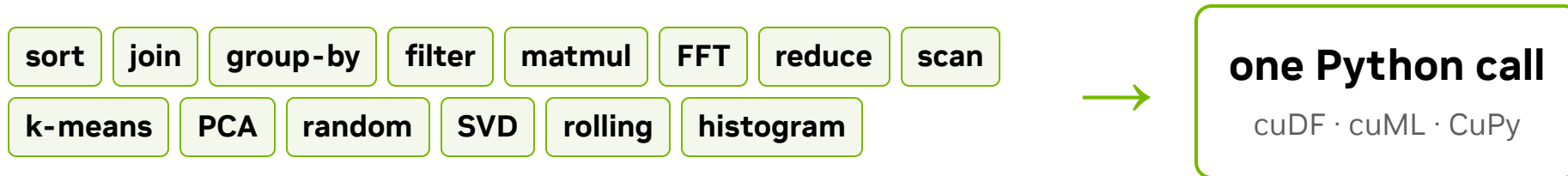
RAPIDS

The **data-science slice** of CUDA-X, open-source, with familiar Python APIs

RAPIDS packages thousands of **optimized GPU kernels** behind APIs you already know:

You already know	RAPIDS gives you	for
<code>pandas</code> · <code>scikit-learn</code> · <code>NumPy</code> · <code>NetworkX</code>	<code>cuDF</code> · <code>cuML</code> · <code>CuPy</code> · <code>cuGraph</code>	dataframes · ML · arrays · graphs

You stand on thousands of tuned kernels



- Each tile is a **GPU kernel** that experts wrote and tuned over years
- RAPIDS bundles them behind the **APIs you already know**
- You write familiar Python; **RAPIDS runs the CUDA for you**

How much faster on real data? You'll measure that yourself in Section 4.

Putting it all together

Your code & notebooks

pandas, scikit-learn, NumPy, your scripts

RAPIDS

cuDF · cuML · cuGraph, the data-science slice of CUDA-X

CUDA Python

cuda.core · cuda.bindings · Numba · CuPy

CUDA-X + CUDA Toolkit

cuBLAS · cuSPARSE · NCCL · libcudart · nvcc

NVIDIA driver

libcuda, the system layer that talks to the hardware

GPU hardware

SMs · warps · thousands of cores · VRAM

Each layer builds on the one below it; your Python sits at the very top.

The mental model for today

- A GPU is **many simple cores + its own fast memory** → built for **parallel, math-heavy** work
- Getting data **onto** the GPU is the slow part → **keep it there**
- **CUDA** (C-based) exposes that power, but you rarely touch it
- **CUDA Python** and the **CUDA Toolkit** are the building blocks
- **RAPIDS / CUDA-X** hand you **ready-made kernels**, so familiar Python code just runs on the GPU



Next: let's get on a GPU